AIDL LAB -16

NAME – SUFIYA AFREEN

ENROLLMENT NO – 2403A51067

BRANCH & BATCH – CSE-03

DATE – 4-11-26

Lab/Exp 16: Transformer Encoder Block

Kaggle Dataset Link:

1. Download and preprocess the IMDb Reviews dataset.

2. Implement the required PyTorch architecture.

3. Train the model using appropriate hyperparameters.

4. Evaluate using suitable performance metrics.

5. Visualize predictions, loss curves, or generated outputs as applicable.

6. Compare results with a baseline model and summarize observations

---

CODE & OUTPUT –

```
# 1. IMPORT LIBRARIES
!pip install -q datasets

import torch
import torch.nn as nn
import torch.optim as optim
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
import random

from collections import Counter
from datasets import load_dataset
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import confusion_matrix

# Reproducibility
torch.manual_seed(42)
np.random.seed(42)
random.seed(42)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)


# 2. DOWNLOAD IMDb DATASET
print("\nDownloading IMDb Dataset...")

dataset = load_dataset("stanfordnlp/imdb")

train_df = pd.DataFrame(dataset["train"])
test_df = pd.DataFrame(dataset["test"])

# Use subsets for faster execution
train_df = train_df.sample(5000, random_state=42)
test_df = test_df.sample(2000, random_state=42)

print("Training samples:", len(train_df))
print("Testing samples:", len(test_df))


# 3. TEXT PREPROCESSING
def clean_text(text):
    text = text.lower()
    text = re.sub(r"<.*?>", "", text)
    text = re.sub(r"[^a-zA-Z\s]", "", text)
    text = re.sub(r"\s+", " ", text)
    return text.strip()

train_df["text"] = train_df["text"].apply(clean_text)
test_df["text"] = test_df["text"].apply(clean_text)
```

```
print("\nText preprocessing completed!")


# 4. CREATE VOCABULARY
MAX_VOCAB = 10000
MAX_LEN = 100

counter = Counter()

for text in train_df["text"]:
    counter.update(text.split())

vocab = {"<PAD>": 0, "<UNK>": 1}

for word, count in counter.most_common(MAX_VOCAB - 2):
    vocab[word] = len(vocab)

print("Vocabulary size:", len(vocab))


# 5. TOKENIZATION
def encode_text(text):
    tokens = text.split()
    ids = [vocab.get(word, 1) for word in tokens[:MAX_LEN]]
    ids += [0] * (MAX_LEN - len(ids))
    return ids

X = np.array([encode_text(t) for t in train_df["text"]])
y = train_df["label"].values

X_test = np.array([encode_text(t) for t in test_df["text"]])
y_test = test_df["label"].values

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)


# 6. PYTORCH DATASET
class IMDBDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.long)
        self.y = torch.tensor(y, dtype=torch.long)

    def __len__(self):
        return len(self.X)

    def __getitem__(self, index):
        return self.X[index], self.y[index]


train_loader = DataLoader(
    IMDBDataset(X_train, y_train), batch_size=64, shuffle=True
)
```

```python
val_loader = DataLoader(
    IMDBDataset(X_val, y_val), batch_size=64
)

test_loader = DataLoader(
    IMDBDataset(X_test, y_test), batch_size=64
)


# 7. TRANSFORMER ENCODER MODEL
class TransformerModel(nn.Module):

    def __init__(self, vocab_size, embed_dim=128,
                 num_heads=4, num_layers=1,
                 ff_dim=256, dropout=0.2):

        super(TransformerModel, self).__init__()

        self.embedding = nn.Embedding(
            vocab_size, embed_dim, padding_idx=0
        )

        self.position_embedding = nn.Embedding(
            MAX_LEN, embed_dim
        )

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embed_dim,
            nhead=num_heads,
            dim_feedforward=ff_dim,
            dropout=dropout,
            batch_first=True
        )

        self.transformer = nn.TransformerEncoder(
            encoder_layer,
            num_layers=num_layers
        )

        self.dropout = nn.Dropout(dropout)
        self.fc = nn.Linear(embed_dim, 2)

    def forward(self, x):

        batch_size, seq_len = x.shape

        positions = torch.arange(
            seq_len, device=x.device
        ).unsqueeze(0).expand(batch_size, seq_len)

        padding_mask = (x == 0)

        x = self.embedding(x) + self.position_embedding(positions)

        x = self.transformer(
            x, src_key_padding_mask=padding_mask
```

```
        x = x / mask.sum(dim=1).clamp(min=1)

        x = self.dropout(x)

        return self.fc(x)


# 8. INITIALIZE MODEL
model = TransformerModel(
    vocab_size=len(vocab)
).to(device)

criterion = nn.CrossEntropyLoss()

optimizer = optim.Adam(
    model.parameters(), lr=0.001
)

EPOCHS = 5

train_losses = []
val_losses = []
val_accuracies = []


# 9. TRAINING FUNCTION
def train_model():

    for epoch in range(EPOCHS):

        model.train()
        total_loss = 0

        for texts, labels in train_loader:

            texts = texts.to(device)
            labels = labels.to(device)

            optimizer.zero_grad()

            outputs = model(texts)
            loss = criterion(outputs, labels)

            loss.backward()
            optimizer.step()

            total_loss += loss.item()

        avg_train_loss = total_loss / len(train_loader)
        train_losses.append(avg_train_loss)

        # Validation
        model.eval()
        val_loss = 0
        correct = 0
        total = 0
```

```
        with torch.no_grad():
            for texts, labels in val_loader:

                texts = texts.to(device)
                labels = labels.to(device)

                outputs = model(texts)

                loss = criterion(outputs, labels)
                val_loss += loss.item()

                predictions = torch.argmax(outputs, dim=1)

                correct += (predictions == labels).sum().item()
                total += labels.size(0)

        avg_val_loss = val_loss / len(val_loader)
        accuracy = 100 * correct / total

        val_losses.append(avg_val_loss)
        val_accuracies.append(accuracy)

        print(
            f"Epoch [{epoch+1}/{EPOCHS}] "
            f"Train Loss: {avg_train_loss:.4f} "
            f"Val Loss: {avg_val_loss:.4f} "
            f"Val Accuracy: {accuracy:.2f}%"
        )


print("\nTraining Transformer Model...")
train_model()


# 10. EVALUATION
model.eval()

all_predictions = []
all_labels = []

with torch.no_grad():

    for texts, labels in test_loader:

        texts = texts.to(device)

        outputs = model(texts)

        predictions = torch.argmax(outputs, dim=1)

        all_predictions.extend(predictions.cpu().numpy())
        all_labels.extend(labels.numpy())

transformer_accuracy = accuracy_score(
    all_labels, all_predictions
)
```

```python
# 10. EVALUATION
model.eval()

all_predictions = []
all_labels = []

with torch.no_grad():

    for texts, labels in test_loader:

        texts = texts.to(device)

        outputs = model(texts)

        predictions = torch.argmax(outputs, dim=1)

        all_predictions.extend(predictions.cpu().numpy())
        all_labels.extend(labels.numpy())

transformer_accuracy = accuracy_score(
    all_labels, all_predictions
)

print("\nTRANSFORMER MODEL RESULTS")
print("Test Accuracy:", round(transformer_accuracy * 100, 2), "%")

print("\nClassification Report:")
print(classification_report(
    all_labels,
    all_predictions,
    target_names=["Negative", "Positive"]
))


# 11. CONFUSION MATRIX
cm = confusion_matrix(all_labels, all_predictions)

plt.figure(figsize=(6, 4))
sns.heatmap(
    cm, annot=True, fmt="d", cmap="Blues",
    xticklabels=["Negative", "Positive"],
    yticklabels=["Negative", "Positive"]
)

plt.title("Transformer Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.show()


# 12. LOSS CURVES
plt.figure(figsize=(8, 5))

plt.plot(train_losses, label="Training Loss")
plt.plot(val_losses, label="Validation Loss")
```

```
plt.title("Transformer Training and Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid()
plt.show()


# 13. VALIDATION ACCURACY GRAPH
plt.figure(figsize=(8, 5))

plt.plot(
    range(1, EPOCHS + 1),
    val_accuracies,
    marker="o",
    label="Validation Accuracy"
)

plt.title("Transformer Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")
plt.legend()
plt.grid()
plt.show()


# 14. BASELINE MODEL: SIMPLE LSTM
class LSTMModel(nn.Module):

    def __init__(self, vocab_size):
        super(LSTMModel, self).__init__()

        self.embedding = nn.Embedding(
            vocab_size, 128, padding_idx=0
        )

        self.lstm = nn.LSTM(
            128, 64, batch_first=True
        )

        self.fc = nn.Linear(64, 2)

    def forward(self, x):

        embedded = self.embedding(x)

        _, (hidden, _) = self.lstm(embedded)

        return self.fc(hidden[-1])


baseline = LSTMModel(len(vocab)).to(device)

baseline_optimizer = optim.Adam(
    baseline.parameters(), lr=0.001
)
```

```python
# Train baseline model
print("\nTraining Baseline LSTM Model...")

for epoch in range(EPOCHS):

    baseline.train()
    total_loss = 0

    for texts, labels in train_loader:

        texts = texts.to(device)
        labels = labels.to(device)

        baseline_optimizer.zero_grad()

        outputs = baseline(texts)

        loss = criterion(outputs, labels)

        loss.backward()
        baseline_optimizer.step()

        total_loss += loss.item()

    print(
        f"Baseline Epoch [{epoch+1}/{EPOCHS}] "
        f"Loss: {total_loss / len(train_loader):.4f}"
    )


# 15. BASELINE EVALUATION
baseline.eval()

baseline_predictions = []
baseline_labels = []

with torch.no_grad():

    for texts, labels in test_loader:

        texts = texts.to(device)

        outputs = baseline(texts)

        predictions = torch.argmax(outputs, dim=1)

        baseline_predictions.extend(
            predictions.cpu().numpy()
        )

        baseline_labels.extend(labels.numpy())

baseline_accuracy = accuracy_score(
    baseline_labels, baseline_predictions
)
```

```
print("\nBASELINE LSTM ACCURACY:",
      round(baseline_accuracy * 100, 2), "%")


# 16. MODEL COMPARISON
comparison = pd.DataFrame({
    "Model": ["LSTM Baseline", "Transformer Encoder"],
    "Accuracy (%)": [
        baseline_accuracy * 100,
        transformer_accuracy * 100
    ]
})

print("\nMODEL COMPARISON")
print(comparison)


plt.figure(figsize=(7, 5))

plt.bar(
    comparison["Model"],
    comparison["Accuracy (%)"]
)

plt.title("LSTM vs Transformer Accuracy")
plt.xlabel("Models")
plt.ylabel("Accuracy (%)")
plt.ylim(0, 100)
plt.show()


# 17. SAMPLE PREDICTIONS
def predict_sentiment(review):

    model.eval()

    review = clean_text(review)
    encoded = encode_text(review)

    tensor = torch.tensor(
        [encoded], dtype=torch.long
    ).to(device)

    with torch.no_grad():

        output = model(tensor)

        prediction = torch.argmax(output, dim=1).item()

    return "Positive" if prediction == 1 else "Negative"


print("\nSAMPLE PREDICTIONS")

print("Review 1:",
      predict_sentiment(
```

```python
print("Review 2:",
      predict_sentiment(
          "This movie is boring and the story is terrible"
      ))


# 18. FINAL RESULTS
print("\n================================")
print("EXPERIMENT 16 COMPLETED")
print("================================")

print("Transformer Accuracy:",
      round(transformer_accuracy * 100, 2), "%")

print("Baseline LSTM Accuracy:",
      round(baseline_accuracy * 100, 2), "%")

if transformer_accuracy > baseline_accuracy:
    print("Transformer performed better than LSTM.")
elif transformer_accuracy < baseline_accuracy:
    print("LSTM performed better than Transformer.")
else:
    print("Both models achieved equal accuracy.")

print("\nAll graphs and evaluation results generated!")
```

```
...  ownloading IMDb Dataset...
     arning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
     ARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
     EADME.md: 100% 7.81k/7.81k [00:00<00:00, 456kB/s]
     ain_text/train-00000-of-00001.parquet: reconstructing file: 100% 21.0MB / 21.0MB, 2.07MB/s
     ain_text/train-00000-of-00001.parquet: downloading bytes: 20.9MB, 2.05MB/s
     ain_text/test-00000-of-00001.parquet: reconstructing file: 100% 20.5MB / 20.5MB, 2.02MB/s
     ain_text/test-00000-of-00001.parquet: downloading bytes: 20.4MB, 2.00MB/s
     ain_text/unsupervised-00000-of-00001.p(...): reconstructing file: 100% 42.0MB / 42.0MB, 4.10MB/s
     ain_text/unsupervised-00000-of-00001.p(...): downloading bytes: 41.9MB, 4.08MB/s
     enerating train split: 100% 25000/25000 [00:00<00:00, 99757.62 examples/s]
     enerating test split: 100% 25000/25000 [00:00<00:00, 108014.53 examples/s]
     enerating unsupervised split: 100% 50000/50000 [00:00<00:00, 91114.75 examples/s]
     aining samples: 5000
     sting samples: 2000

     xt preprocessing completed!
     cabulary size: 10000

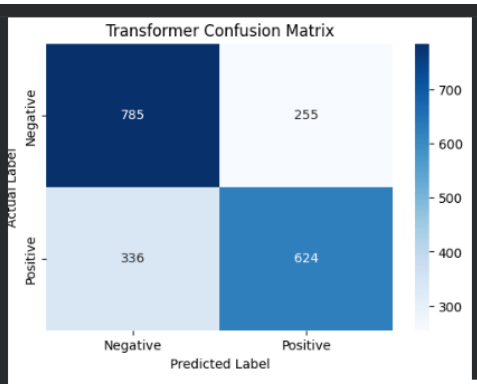
     aining Transformer Model...
     sr/local/lib/python3.13/dist-packages/torch/nn/modules/transformer.py:531: UserWarning: The PyTorch API of nested tensors is in prototype stage and will change in the near future. We recommend specifying layout=torch.jagged
      output = torch._nested_tensor_from_mask(
     och [1/5] Train Loss: 0.6898 Val Loss: 0.6471 Val Accuracy: 63.00%
     och [2/5] Train Loss: 0.5930 Val Loss: 0.5892 Val Accuracy: 68.70%
     och [3/5] Train Loss: 0.4548 Val Loss: 0.5860 Val Accuracy: 71.80%
     och [4/5] Train Loss: 0.2952 Val Loss: 0.6853 Val Accuracy: 72.90%
     och [5/5] Train Loss: 0.1502 Val Loss: 0.8783 Val Accuracy: 72.20%

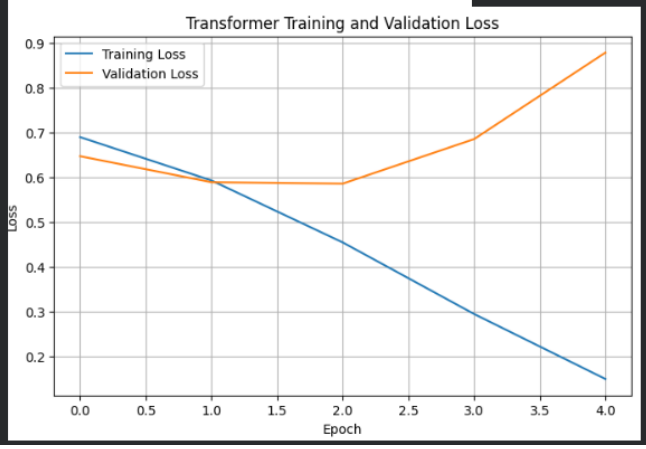
     ANSFORMER MODEL RESULTS
     st Accuracy: 70.45 %

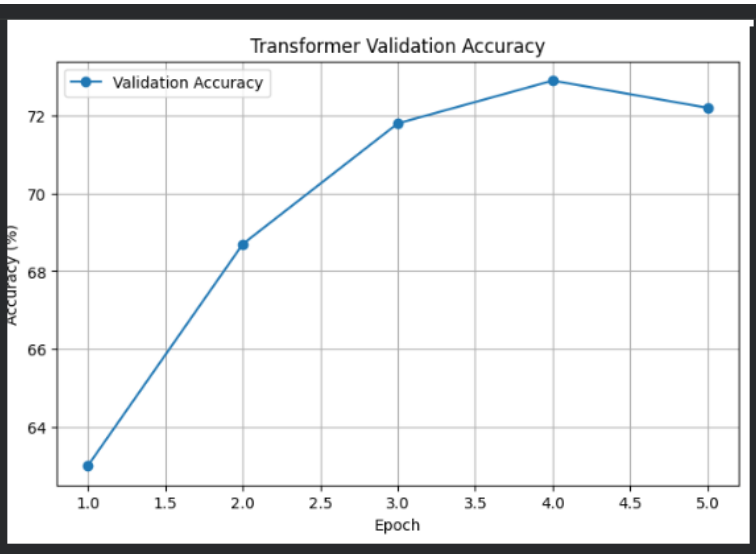
     lassification Report:
                   precision    recall  f1-score   support

        Negative       0.70      0.75      0.73      1040
        Positive       0.71      0.65      0.68       960

        accuracy                           0.70      2000
       macro avg       0.71      0.70      0.70      2000
     ighted avg       0.70      0.70      0.70      2000
```



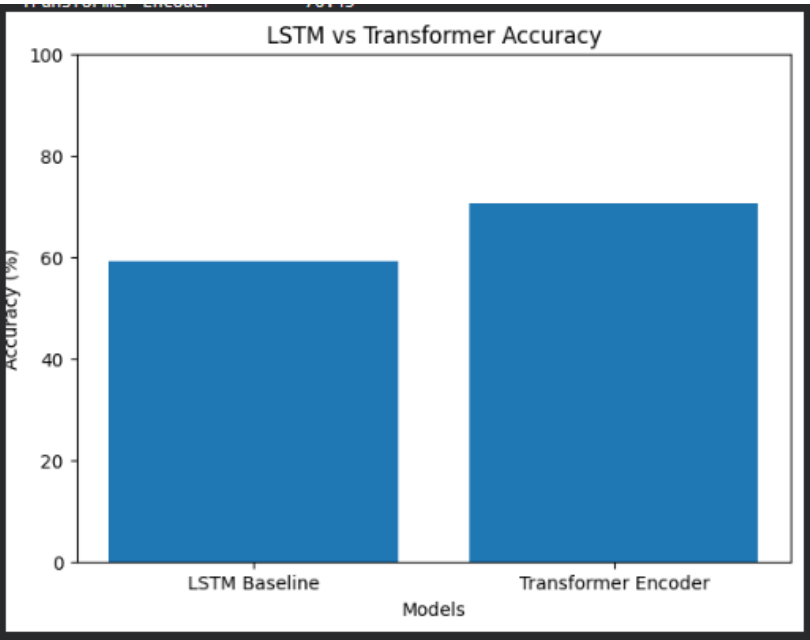





```
raining Baseline LSTM Model...
aseline Epoch [1/5] Loss: 0.6947
aseline Epoch [2/5] Loss: 0.6731
aseline Epoch [3/5] Loss: 0.6413
aseline Epoch [4/5] Loss: 0.5609
aseline Epoch [5/5] Loss: 0.4620

SELINE LSTM ACCURACY: 59.25 %

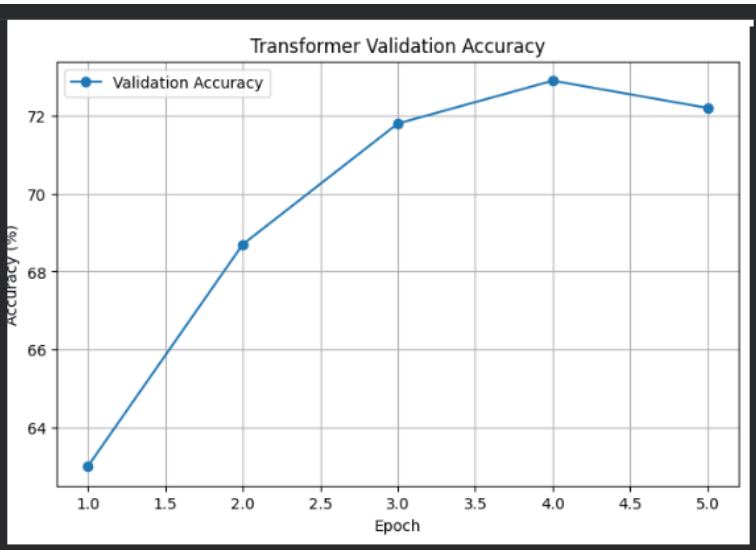
DEL COMPARISON
                Model  Accuracy (%)
        LSTM Baseline         59.25
  Transformer Encoder         70.45
```




```
MPLE PREDICTIONS
eview 1: Positive
eview 2: Negative

==============================
(PERIMENT 16 COMPLETED
==============================
ransformer Accuracy: 70.45 %
aseline LSTM Accuracy: 59.25 %
ransformer performed better than LSTM.

ll graphs and evaluation results generated!
```




```
aining Baseline LSTM Model...
aseline Epoch [1/5] Loss: 0.6947
aseline Epoch [2/5] Loss: 0.6731
aseline Epoch [3/5] Loss: 0.6413
aseline Epoch [4/5] Loss: 0.5609
aseline Epoch [5/5] Loss: 0.4620

SELINE LSTM ACCURACY: 59.25 %

DEL COMPARISON
                Model  Accuracy (%)
        LSTM Baseline         59.25
  Transformer Encoder         70.45
```